

Sujet $\mathcal{F}$

□ Exercice : type A

On s'intéresse dans cet exercice à la compression de chaînes de caractères représentant des séquences génétiques codées sur l'alphabet $\{A, C, G, T\}$ avec l'algorithme de Huffman. On rappelle que le principe de cet algorithme est d'attribuer aux caractères les plus fréquents un code plus court.

1. On veut compresser la séquence $S = \text{ATGTGATGTCCT}$, donner le nombre d'occurrences de chaque caractère dans cet séquence.
2. Construire l'arbre de Huffman associé à la compression de S .
3. Donner les codes obtenus pour chacun des quatre caractères A, C, G et T . En déduire le taux de compression de l'algorithme. On supposera qu'initialement la séquence est codée en ASCII et que donc chacun des caractères occupe un octet et on ne tiendra pas compte de la taille de l'arbre.
4. Quelle structure de données abstraite est utilisée pour implémenter cet algorithme ? Quelle en est l'implémentation usuelle ?
5. Proposer un type en OCaml permettant de représenter un arbre de Huffman.

□ Exercice : type B

On dispose d'un *système monétaire* c'est-à-dire d'un ensemble de valeurs possibles pour les pièces et les billets. Le problème du rendu de monnaie consiste à déterminer le nombre minimal de pièces à utiliser pour former une somme donnée. On supposera que les valeurs des pièces et des billets sont des entiers de même que la somme à rendre et que le système monétaire contient toujours la valeur 1 (de cette façon, le problème admet toujours une solution). Les fonctions demandées dans cet exercice sont à écrire en OCaml et on donnera le système monétaire sous la forme de la liste *triée dans l'ordre décroissant* des valeurs des pièces. Par exemple, la liste $[500; 200; 100; 50; 20; 10; 5; 2; 1]$ est un système monétaire correct et le nombre minimal de pièces pour rendre la somme 17 est 3 (obtenue en utilisant $10 + 5 + 2$). D'autre part, on s'interdit dans cet exercice l'utilisation des aspects impératifs du langage OCaml (en particulier les boucles et les références.)

1. Ecrire une fonction `verifie : int list -> bool` qui prend en argument un système monétaire et renvoie un booléen indiquant si ce système est valide (c'est à dire la liste des valeurs est rangée dans l'ordre décroissant et se termine par 1).
2. On considère dans un premier temps la méthode consistant à rendre toujours la pièce de plus forte valeur possible. A quelle famille d'algorithme appartient cette méthode ? Justifier.
3. En utilisant un exemple de votre choix, montrer que cette méthode ne fournit pas toujours le nombre minimal de pièces.
4. Donner une implémentation de cette méthode sous la forme d'une fonction `monnaie : int list -> int -> int` prenant en argument un système monétaire ainsi qu'une somme et renvoyant le nombre de pièces.
5. On veut maintenant résoudre ce problème par programmation dynamique. On note $(p_i)_{0 \leq i \leq n}$ les valeurs des pièces rangées dans l'ordre décroissant, et on note $m(S, k)$ le nombre minimal de pièces pour rendre la somme S en utilisant seulement les pièces $p_k, \dots, p_{n-1}$. On convient que $m(S, k) = +\infty$ si $S \neq 0$ et $k \geq n$ car on cherche alors à rendre une somme non nulle sans utiliser de pièces. Donner la relation liant $m(S, k)$ à $m(S, k+1)$ si on choisit de ne pas utiliser la pièce p_k . De même trouver une relation entre différentes instances du problème lorsqu'on choisit d'utiliser p_k (dans ce cas, on a nécessairement $S \geq p_k$).
6. Ecrire une fonction `dynamique : int list -> int -> int` qui résout le problème par programmation dynamique, on pourra utiliser la valeur `Int.max` de OCaml afin d'indiquer que la résolution est impossible (nombre infini de pièces).